Intelligensi.Deploy New Build Requirements

Purpose

Build a new version of Intelligensi.Deploy as a clear, non-technical model
deployment product.

The product should let an operator choose an AI model, choose where to run it,
press one large deploy button, watch understandable progress logs, preview the
deployed model, and receive guided recovery if deployment fails.

The current dashboard has useful deployment logic, presets, Lambda availability
checks, logs, preview plumbing, and repair classification, but the interface has
become too operational and fragile. The new build should preserve the useful
backend concepts while replacing the interface with a guided workflow.

Strategic Product Direction

Intelligensi.Deploy should evolve beyond a deployment dashboard into a curated
AI runtime platform for deploying, previewing, operating, and consuming
generative AI models through standardized workflows.

The product should abstract infrastructure complexity away from users and focus
the primary experience on the outcomes users care about:

- deployable AI runtimes
- guided previews
- temporary inference sessions
- runtime healing
- workflow portability
- verified deployment profiles
- standardized AI appliance experiences

The long-term product direction is:

1. Deploy a model.
2. Use the model immediately through a generated UI.
3. Shut the model down automatically when finished.

Users should interact primarily with:

- prompts

- workflows
- previews
- outputs
- costs
- deployment health

Users should not need to understand:

- Docker
- SSH
- CUDA
- inference servers
- cloud GPU infrastructure
- container orchestration

The platform should progressively hide operational complexity while retaining optional advanced controls for technical users. Advanced controls should be available when needed, but they should not be required for the happy path.

This means the product should not present itself as a generic cloud deployment console. It should present itself as a runtime control plane for AI workloads, where the user chooses what they want to run, sees whether it is ready, deploys it, uses it, and receives guided recovery if anything fails.

Runtime Profile Architecture

The platform should standardize deployable AI runtimes through a normalized runtime profile system.

The platform should not treat Docker images as isolated technical artifacts. Instead, each deployable runtime should represent a complete verified AI appliance. A runtime profile should describe not only how to start a container, but also what the runtime does, how it is previewed, what inputs it accepts, what outputs it produces, how health is checked, how failures are repaired, and how confident the platform is that deployment will succeed.

Each runtime profile may include:

- model
- workflow

- runtime image
- preview schema
- health contract
- deployment requirements
- GPU requirements
- startup lifecycle
- inference capabilities
- supported inputs
- supported outputs
- repair strategies
- deployment confidence score

Example runtime categories:

- Flux Fashion Studio
- LTX Music Video Generator
- DeepSeek Coding Assistant
- Whisper Audio Transcriber

The workflow itself may be more commercially important than the underlying base
model. For example, a packaged video generation workflow with the right preview
form, prompt controls, seed image handling, resolution options, health checks,
and repair recipes may be more valuable to a user than simply exposing the raw
model name.

Runtime profiles should therefore become the core product object. Hugging Face
models, Docker images, GitHub repositories, presets, and provider settings
should be normalized into runtime profiles before they appear in the user
interface.

Minimum runtime profile requirements for the first build:

- stable runtime ID
- display name
- plain-language description
- model source
- runtime image or deployment preset
- supported provider, initially Lambda
- recommended instance type
- required secrets and environment variables
- startup command
- health check path or command
- preview schema
- estimated hourly cost where known
- verification level

- deployment confidence score

ComfyUI Runtime Export Pipeline

ComfyUI workflows should become first-class deployable assets.

Target pipeline:

```text
ComfyUI Workflow
    ↓
Workflow Validator
    ↓
Inference Schema Generator
    ↓
Container Builder
    ↓
Runtime Profile
    ↓
Deployable Marketplace Item
```

The export pipeline should:

- validate required nodes
- detect incompatible or custom nodes
- generate preview schemas automatically
- estimate GPU requirements
- generate health contracts
- build standardized inference containers
- produce deployment metadata
- generate deployment confidence scoring

The resulting runtime should be reproducible and portable across providers. A workflow exported from ComfyUI should not remain a local-only graph that requires manual interpretation. It should become a packaged runtime with a clear preview UI, defined inputs, predictable outputs, known hardware requirements, and a repeatable deployment path.

The first implementation does not need to build the full ComfyUI exporter, but the data model should leave room for it. Runtime profiles should be able to reference a source workflow, generated inference schema, container build metadata, required custom nodes, and portability warnings.

For the first build, ComfyUI runtime export can be represented as:

- manual or scripted workflow import
- validation report
- generated preview schema
- generated runtime profile
- deployment readiness score
- unsupported node warnings

Preview-Driven Architecture

Preview is a core product capability and a potential standalone commercial
product.

The preview layer should:

- dynamically generate user interfaces from runtime metadata
- provide temporary inference sessions
- support deploy-use-shutdown workflows
- store preview history
- provide artifact download and sharing
- expose only workflow-relevant controls

The preview schema should drive:

- UI generation
- validation
- inference payload generation
- runtime capability discovery
- deployment compatibility
- preview persistence
- workflow portability

The preview layer should eventually support:

- image generation studios
- video generation studios
- LLM playgrounds
- embedding explorers
- audio generation and transcription interfaces
- workflow templates
- collaborative sessions

The first build should use preview schemas to generate practical forms for each
runtime. For example, an LTX 2.3 image or video runtime may expose prompt, seed
image, resolution, duration, guidance, and seed controls. A DeepSeek runtime may
expose a prompt, system instruction, temperature, max tokens, and conversation
history.

Preview should only become available after a deployment is healthy, unless the
runtime supports a local or mocked preview mode. If deployment fails, the preview
area should clearly explain that the runtime is not available and link the user
to the log and healing recommendation.

Preview sessions should be designed as temporary inference sessions. The user
should be encouraged to deploy, generate the needed outputs, download or share
the artifacts, and shut the runtime down when finished. The interface should
make cost visible throughout this flow.

Runtime Verification Levels

The platform should support runtime trust tiers.

Initial trust levels:

- verified
- community
- experimental
- unsafe

Definitions:

`verified`

- fully tested by Intelligensi
- deployment and preview validated
- repair flows available
- recommended for production usage

`community`

- community-submitted runtime
- partially validated
- limited guarantees
- suitable for users who accept some operational risk

`experimental`

- unstable or partially working runtime
- limited healing support
- useful for testing, research, and early integrations

`unsafe`

- unsupported or unverified runtime
- manual deployment risk accepted by user
- should require explicit confirmation before deployment

The verification system should become a core trust and ecosystem mechanism. It
should influence search ranking, deployment warnings, preview availability,
repair confidence, and marketplace eligibility.

Runtime cards should show verification level clearly, but without overwhelming
non-technical users. The UI should translate verification into plain language,
such as `Ready`, `Community tested`, `Experimental`, or `Manual risk`.

Healing Intelligence Layer

Healing and deployment recovery are strategic intellectual property.

The healing layer should:

- classify deployment failures
- detect recurring runtime patterns
- recommend bounded safe repairs
- learn from deployment outcomes
- accumulate provider reliability metrics
- accumulate runtime compatibility intelligence
- improve deployment confidence scoring over time

The healing system should evolve into:

- runtime repair intelligence
- deployment optimization
- provider recommendation intelligence
- startup failure prediction
- automated recovery orchestration

The platform should treat deployment telemetry as structured operational
intelligence. Logs should not only be displayed to users; they should also be
classified into machine-readable events that can improve future deployments.

Examples of structured healing events:

- missing required secret
- invalid Hugging Face token
- Docker image unavailable

- image pull authentication failure
- CUDA mismatch
- insufficient GPU memory
- runtime port did not open
- health check timeout
- model download failed
- Lambda capacity unavailable
- startup command exited early

The first build should support deterministic diagnosis for known failure cases
and explain them in plain language. Later builds should use accumulated
deployment outcomes to recommend better instances, better provider regions,
runtime-specific fixes, and pre-deployment warnings.

Repairs should be bounded and safe. The system may apply automatic fixes only
when the action is low risk and clearly understood, such as adding a missing
non-secret environment variable from a verified profile, retrying a transient
provider error, or switching to an equivalent available instance type after user
confirmation.

Open Source Strategy

The platform should support an open-core development model.

Potential open-source scope:

- UI shell
- workflow schemas
- provider adapters
- deployment orchestration
- runtime profile specifications
- local deployment support
- community runtime registry

Potential proprietary or commercial scope:

- healing intelligence
- runtime verification pipeline
- deployment optimization
- warm-pool orchestration
- deployment telemetry intelligence
- runtime scoring
- commercial runtime marketplace
- enterprise operational tooling

The open-source ecosystem should encourage:

- runtime contributions
- provider integrations
- workflow packaging
- preview schemas
- deployment profiles
- repair recipes

The open-core model should make it easy for developers and model creators to add
new runtime profiles while allowing Intelligensi to build a defensible commercial
layer around verification, healing, optimization, telemetry intelligence, and
managed runtime operations.

Product Goals

- Make deployment understandable for non-technical users.
- Reduce the primary workflow to: choose model, choose platform/instance,
  deploy, preview, fix if needed.
- Make logs visible by default during every deployment.
- Detect missing or incorrect settings before deployment starts.
- Use model metadata to generate the correct preview form.
- Provide self-healing guidance and safe automatic repair where possible.
- Start with Lambda as the only deployment platform, but design the model so
  Nebius, GCP, and other providers can be added later.

Technology Requirements

- Frontend: TypeScript, React, Tailwind CSS.
- App structure: component-based workflow UI.
- State: local-first state persistence for settings, selected model, selected
  platform, deployment state, logs, and preview config.
- Backend/control plane: TypeScript preferred for the new build.
- Existing scripts, presets, and service containers may be reused where they are
  still reliable.
- Secrets must not be committed to git or exposed in browser local storage.
- Logs must be written to an operator-visible deployment log and streamed to the
  UI.

Primary User Workflow

1. User opens Intelligensi.Deploy.

2. User searches or browses available models.
3. User selects a model from a Hugging Face scanned list.
4. System shows what the model does, what hardware it needs, and whether a
   compatible Docker image or preset exists.
5. User chooses a deployment platform.
6. Initial platform scope is Lambda only.
7. User chooses an available Lambda instance type.
8. System validates required settings.
9. User clicks a large `Deploy` button.
10. UI opens the deployment log automatically.
11. System streams every deployment step and validation result.
12. If deployment succeeds, a model-specific preview panel becomes available.
13. User submits a preview request.
14. UI displays the result and stores preview history.
15. If deployment fails, the self-healing agent explains the failure and either
    applies a safe fix or gives the exact manual fix required.

Main Screens

1. Model Selection

This is the first screen, not an admin dashboard.

Requirements:

- Search Hugging Face models.
- Show a curated/scored top list when search is empty.
- Show model cards with:
  - model name
  - task type
  - source/provider
  - downloads/likes when available
  - model family
  - deployment readiness
  - linked GitHub or Docker image when available
  - required secrets, such as Hugging Face token
  - recommended hardware
- Mark models as:
  - ready to deploy
  - needs Docker image
  - needs token
  - experimental
  - unsupported
- Use fallback curated data if Hugging Face scanning is unavailable.
- Fallback data must be clearly marked.

2. Model Detail / Deployment Setup

```
After a model is selected, show a simple setup screen.

Requirements:

- Show model purpose in plain language.
- Show deployment requirements:
  - Docker image
  - port
  - health path
  - environment variables
  - required tokens
  - minimum GPU memory
  - recommended Lambda instance type
- Show deployment target:
  - Platform: Lambda initially
  - Region
  - Instance type
  - Estimated cost per hour
  - Availability status
- Hide low-level fields by default.
- Provide an `Advanced settings` section for technical users.
- Save selected settings locally.
- Validate required settings before deployment.

### 3. Deployment Screen

This screen opens automatically after `Deploy`.

Requirements:

- A single large deploy CTA.
- When clicked, immediately write a deployment event to the log.
- Open the log panel automatically.
- Stream all activity:
  - frontend click received
  - settings validation
  - selected model
  - selected Docker image
  - selected Lambda region and instance type
  - availability check
  - instance launch request
  - SSH readiness
  - Docker pull/build
  - container start
  - health checks
  - preview endpoint readiness
  - failure classification
  - repair actions
- Logs must use human-readable language.
- Raw technical logs can be shown under an expandable `Technical
details`
  section.
```

- The user must never be left with a silent button click.

4. Preview Screen

Preview becomes available only after a deployment is healthy.

Requirements:

- The preview form is generated from model/task metadata.
- Image generation models:
  - prompt
  - negative prompt when supported
  - resolution
  - seed
  - steps/quality mode
- Video generation models such as LTX 2.3:
  - prompt
  - optional seed image
  - resolution
  - duration/frames
  - quality mode
  - guidance/steps where supported
- LLM models such as DeepSeek:
  - prompt
  - system prompt if supported
  - temperature
  - max tokens
- Embedding models:
  - text input
  - output vector preview or similarity test
- Preview output should show:
  - result
  - model used
  - endpoint used
  - render/inference time
  - key settings
  - link to generated artifact when applicable

5. Failure And Self-Healing

If deployment fails, the UI should change from deployment progress to recovery.

Requirements:

- Classify failures from logs and structured errors.
- Show a simple failure summary first.
- Show evidence from logs second.
- Show exact next action third.
- Safe automatic repair is allowed for:
  - Lambda capacity fallback
  - refreshing instance type availability

```
  - SSH readiness retry
  - health check retry with longer timeout
  - one container restart
  - one Docker rebuild or pull retry
- The system must not automatically:
  - overwrite secrets
  - delete cloud instances without confirmation
  - change billing-sensitive provider settings without
confirmation
  - run destructive commands
  - retry endlessly
- Manual-only failures include:
  - missing Lambda API key
  - invalid Lambda API key
  - missing GHCR token
  - missing Hugging Face token for gated model
  - missing SSH key
  - unavailable Docker image
  - model license/access blocked
- Recovery UI should include:
  - suggested fix
  - whether the fix can be auto-applied
  - `Apply fix and retry`
  - `Retry`
  - `Edit settings`
  - `Cancel deployment`

## Model Metadata Requirements

Each deployable model should resolve to a normalized deployment
profile.

Required fields:

```text
ModelDeploymentProfile
- id
- displayName
- source
- huggingFaceModelId
- taskType
- suitability
- description
- dockerImage
- dockerImageSourceUrl
- githubRepoUrl
- providerSupport
- requiredSecrets
- requiredEnv
- defaultEnv
- port
- healthPath
```
```

```
- previewSchema
- minimumGpuMemoryGb
- recommendedInstanceTypes
- estimatedCostPerHour
- readinessStatus
- warnings
```

Task types:

- image_generation
- video_generation
- llm_inference
- embeddings
- audio_transcription
- custom

Readiness statuses:

- ready
- needs_secret
- needs_docker_image
- needs_model_access
- experimental
- unsupported

Initial Model Examples

LTX 2.3

- Task: video generation.
- Model: `Lightricks/LTX-2.3`.
- Docker image: `ghcr.io/intelligensi-ai/ltx-2.3-worker:experimental`.
- Initial platform: Lambda.
- Recommended hardware: A100 class or better.
- Preview input:
  - prompt
  - optional seed image
  - resolution
  - duration/frames
  - quality mode
- Status: experimental until real inference is wired and smoke-tested.

LTX Video Worker

- Task: video generation.
- Model: `Lightricks/LTX-Video`.
- Docker image: `ghcr.io/intelligensi-ai/ltx-worker:latest`.
- Initial platform: Lambda.
- Recommended hardware: A10 or better.

- Preview input:
  - prompt
  - optional seed image if supported
  - resolution
  - duration/frames
  - quality mode

Flux

- Task: image generation.
- Model: `black-forest-labs/FLUX.1-schnell`.
- Docker image:
  `ghcr.io/intelligensi-ai/intelligensi-image-server:latest`.
- Initial platform: Lambda.
- Recommended hardware: A10 or better.
- Preview input:
  - prompt
  - resolution
  - seed/settings where supported

DeepSeek

- Task: LLM inference.
- Model source: Hugging Face or compatible runtime.
- Preview input:
  - prompt
  - system prompt when supported
  - temperature
  - max tokens
- Deployment profile requires a compatible runtime image before it can be marked
  ready.

Platform Requirements

Lambda Initial Scope

The first production path should support Lambda only.

Requirements:

- Save Lambda API key securely in local secret storage.
- Query Lambda instance availability.
- Show available regions and GPU types.
- Recommend a suitable instance based on model requirements.
- Re-check availability immediately before launch.
- Handle insufficient capacity by trying the next ranked compatible option.
- Show hourly cost.
- Store selected platform settings locally.
- Stop and explain when no compatible instance is available.

Future platforms:

- Nebius.
- GCP.
- Existing/manual GPU VM.
- Hugging Face endpoints if appropriate.

Logging Requirements

Logging is a core product feature, not an afterthought.

Requirements:

- Every user action that can start a deployment must write a log event.
- Every backend deployment stage must write a log event.
- Logs must stream into the UI.
- Logs must persist locally.
- The log view must open automatically during deployment.
- The log view must show:
  - timestamp
  - stage
  - plain-English message
  - severity
  - optional raw details
- The UI must show a visible error if log streaming fails.
- There must be no silent launch failures.

Required deployment stages:

```text
clicked
validated_settings
checked_availability
selected_instance
launch_requested
instance_booting
ssh_waiting
ssh_ready
docker_pull_or_build
container_starting
health_checking
healthy
preview_ready
failed
repair_suggested
repair_applied
retrying
cancelled
```

Local Persistence Requirements

Local-first persistence is required before Supabase.

Requirements:

- Persist selected model.
- Persist selected platform.
- Persist selected Lambda region and instance type.
- Persist non-secret advanced settings.
- Persist deployment history.
- Persist preview history.
- Persist repair history.
- Persist logs.
- Persist secrets only in local secret storage, not browser local storage.
- Refreshing the page must not wipe entered settings.

Storage options:

- Browser localStorage for non-secret UI drafts.
- Local JSON files for deployment config and local runtime state.
- Local secret JSON files only for operator convenience during development.
- Later: Supabase for shared team state, audit trail, and deployment history.

Supabase Future Scope

Supabase is not required for the first rebuild, but the data model should be
compatible with it.

Candidate tables:

- users
- workspaces
- model_profiles
- deployment_profiles
- deployments
- deployment_events
- provider_credentials
- preview_runs
- repair_actions
- audit_events

Secrets should use a proper secret manager or encrypted storage, not plaintext
database rows.

Non-Technical UX Requirements

- Use plain language.

- Avoid raw endpoint URLs unless copied or expanded.
- Avoid showing shell commands by default.
- Use clear states:
  - Ready
  - Needs setting
  - Checking
  - Deploying
  - Healthy
  - Preview ready
  - Failed
  - Fix available
- Use one obvious primary action per step.
- Hide advanced settings until requested.
- Make missing settings actionable.
- Make costs visible before deploy.
- Make cloud billing stop/cancel actions explicit.

Out Of Scope For First Build

- Multi-user Supabase deployment state.
- Full provider marketplace.
- Automatic Docker image creation for arbitrary Hugging Face models.
- Fully autonomous destructive repair actions.
- Production secret management.
- Billing reconciliation.
- Complex workflow builder.

Suggested Build Phases

Phase 1: Product Skeleton

- Create TypeScript React Tailwind app.
- Add route/layout structure.
- Add local state model.
- Add guided workflow screens.
- Add local persistence for drafts.
- Add mock model profiles.

Phase 2: Model Discovery

- Add Hugging Face scanning.
- Add curated fallback profiles.
- Add normalized `ModelDeploymentProfile`.
- Add model readiness scoring.
- Add Docker/GitHub metadata links.

Phase 3: Lambda Deployment Path

- Add Lambda config.
- Add Lambda availability.
- Add instance recommendation.

- Add deploy button.
- Add deployment event stream.
- Reuse existing Lambda launch logic where reliable.

Phase 4: Preview

- Add task-specific preview schema.
- Add preview forms for image, video, and LLM models.
- Add preview result history.
- Add endpoint health gating.

Phase 5: Self-Healing

- Add failure classification.
- Add repair suggestions.
- Add bounded safe repair actions.
- Add manual action UI.
- Add retry flow.

Phase 6: Production Hardening

- Replace local secret storage with secure secret management.
- Add structured deployment event store.
- Add tests for all workflows.
- Add real launch smoke tests.
- Prepare Supabase integration if team/shared state is required.

Acceptance Criteria

- A non-technical user can deploy a ready model without reading a shell command.
- Selecting a model automatically shows compatible platform and instance choices.
- Missing settings are shown before deployment starts.
- Pressing `Deploy` always produces visible log feedback.
- Deployment logs stream until success or failure.
- Successful deployments unlock a model-specific preview.
- Failed deployments show a classified cause and next action.
- Safe repairs are bounded and visible.
- Refreshing the page does not lose settings.
- Lambda is the first fully supported provider.